Chapter 15: Moving to a SQL Server

SQLite has already done useful work for us. We have imported IMDb data, compared indexes, and connected a database to a Python application. A database file is a good fit for that work. Now we will keep the same kind of data and move the storage into a separate server process.

This is a catch-up chapter. We will get the IMDb files, install PostgreSQL in a GitHub Codespace, and import a movie sample. Then we will connect a small Streamlit dashboard and add queries that summarize and compare movies. The interface stays small. The SQL does the interesting work.

Three Lectures

Meeting	Focus	Working result
Wednesday, October 28	Get the IMDb files; install PostgreSQL; prepare CSV and import tables with indexes	Verified row counts and a query plan
Monday, November 2	Connect the data layer and Streamlit; add grouping and aggregates	Search and yearly-summary dashboard views
Wednesday, November 4	Add ranking, previous-row comparisons, and running totals	Window-function views and completed exercises

The same dataset and application carry through all three meetings. The supplied code provides a working baseline; the exercises change the queries and display their results.

The Result We Are Building

Page	Operation	Result
Search: By Name	Filter and join	Movies matching part of a title
Search: By Year	Filter by release year	Movies from the selected year
Search: By Rating	Filter by an inclusive rating range	Movies within the selected range
Notes	Create, read, update and delete	Course notes for a selected movie
Reports	Grouping and window calculations	Charts of ratings, counts, running totals and yearly differences

The progression is data files, PostgreSQL tables, SQL results, and a Streamlit view. The supplied `database.py` owns the queries. `app.py` chooses controls and displays results. A chart does not need to know how PostgreSQL stores a row.

A Database Server Is a Separate Process

With SQLite, the Python library opens a file. With PostgreSQL, Python connects to a running server. The server owns the data files and accepts requests from clients such as the `psql` command-line program or a Python driver. PostgreSQL is an open-source relational database system. (1)

SQLite exercise	PostgreSQL exercise
Open `imdb.db`	Connect to database `imdb`
Python uses `sqlite3`	Python uses Psycopg
SQL parameters use `?`	Psycopg parameters use `%s`
The application opens the database file	The server opens its storage; clients send requests

The server in this exercise runs inside the same Codespace as Python. A server does not have to be on another computer. Separate process ownership and a client connection are the important differences here.

Get the IMDb Data First

IMDb supplies compressed tab-separated values (TSV), rather than comma-separated values (CSV). The first row gives column names, and `\N` marks a missing value. We need `title.basics.tsv.gz` and `title.ratings.tsv.gz`. The files change over time, so record counts will vary. (2)

Work from `chapter-15-moving-to-a-sql-server/code`:

```
cd chapter-15-moving-to-a-sql-server/code
bash download-imdb.sh
```

The script downloads both archives into `data/`, then checks that they are valid gzip files. It leaves existing nonempty files in place. If the earlier optimization exercise already downloaded them, copy those two files into this `data` directory before running the check. Keep them compressed.

IMDb supplies these files for personal and non-commercial use under its published terms. Keep your downloaded data local to the exercise rather than adding it to a public repository. (2) The supplied `.gitignore` excludes `data/` and private database storage. Check the ignore rule before committing:

```
git check-ignore -v data/title.basics.tsv.gz
```

The expected result is a matching ignore rule. Downloading the full archives can take a while even though we will import only a sample. Check available Codespace disk space with `df -h .` first.

Install and Start PostgreSQL in the Codespace

These scripts target an Ubuntu-based Codespace. They install PostgreSQL from Ubuntu's package repositories; this exercise does not need a separate vendor repository or signing key. The package version depends on the Ubuntu release. (3)

```
bash install-postgres.sh
bash start-postgres.sh
```

The first script updates the package list and installs the server and client. The second creates a private PostgreSQL cluster under `.private-postgres` and starts it. Here, *cluster* means a PostgreSQL storage installation containing databases; it does not imply several machines.

The private exercise cluster uses port 5433 rather than PostgreSQL's default 5432, so a package-created default cluster can coexist with it. The startup script creates a login role named `student` and a database named `imdb`. Its final query should report `imdb` as the current database and `student` as the current user. Running the script again leaves the stored data in place.

This is a personal exercise server. It accepts password-free connections only on the local loopback address, `127.0.0.1`. Do not forward PostgreSQL's port, 5433. The Streamlit web interface will use a different port. The later account exercise separates the setup role from the dashboard login. The loopback-only configuration remains a personal classroom setup.

Check the server independently of Python:

```
psql 'postgresql://student@127.0.0.1:5433/imdb' \
  -c 'SELECT version();'
```

A printed server version confirms that `psql` reached a running server. Merely having `psql` installed does not establish that.

After stopping and resuming the Codespace, run `bash start-postgres.sh` again. The private storage directory is inside the workspace, so it can survive a container rebuild, but installed packages may need reinstalling. Deleting the entire Codespace deletes its local database storage. This chapter uses an explicit startup command rather than changing the whole course's Codespace configuration.

If another PostgreSQL server already occupies port 5433, the private server will not start. Check its log at `.private-postgres/server.log` and identify the existing service before continuing. Do not delete its files to make room for this exercise.

Prepare Portable CSV Files

Create a Python virtual environment and install the exercise dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
export DATABASE_URL='postgresql://student@127.0.0.1:5433/imdb'
python prepare_csv.py --start-year 1975 --end-year 1999 --limit 0
```

The preparation script selects movies released from 1975 through 1999, inclusive, whose adult flag is zero. The year range is also the command-line default. It writes `titles.csv` and the corresponding `ratings.csv`. The CSV files use a header row and ordinary CSV quoting, so a comma in a title does not become another column. Missing numeric values become empty CSV fields and will import as SQL `NULL`.

The selection covers the last quarter of the twentieth century, not all film history. Some movies have no matching published rating. The command uses `--limit 0` to keep every eligible movie within the year range. A positive limit takes the first matching records in source-file order and can omit familiar titles. The script holds selected title identifiers in memory so it can select matching ratings; memory use grows with that selection.

CSV file	Columns
`titles.csv`	`title_id`, `primary_title`, `start_year`, `runtime_minutes`, `genres`
`ratings.csv`	`title_id`, `average_rating`, `num_votes`

These CSV files are also suitable inputs for another SQL database or a MongoDB importer with the appropriate numeric conversion. CSV does not carry a database schema or type declarations by itself.

Import the Tables and Add Indexes

```
python import_imdb.py
```

The importer creates the dedicated `imdb_course` schema. A schema is a namespace for related database objects. It creates two tables, streams the CSV files through PostgreSQL's `COPY` interface, and adds indexes after loading. `COPY FROM STDIN` receives data from the client, so the server does not need to open a filename on the client's filesystem. (4)

Both tables use `title_id` as a primary key. The ratings table also has a foreign key referring to titles. Keeping only matching ratings during preparation makes that relationship explicit. The importer runs as one transaction: the new tables and loaded data commit together.

If `imdb_course` already exists, the script stops. To deliberately rebuild this exercise schema:

```
python import_imdb.py --replace
```

That option replaces only `imdb_course`, including objects you have added inside it. Keep unrelated work in another schema. Input files are checked before replacement begins, and a failed import rolls back the schema changes.

The secondary indexes support concrete query patterns:

Index	Query it can help
`titles_year_title`	Filter on a year and order titles within that year
`titles_primary_title`	Equality lookup on the title
`ratings_votes`	Read ratings in descending vote order

Primary keys already supply indexes on `title_id`. The loader also runs `ANALYZE`, which collects table statistics for the planner. A suitable index gives the planner another choice; it does not require every query to use it. (5)

The dashboard's substring search uses `ILIKE '%text%'`. The ordinary title index is included for equality experiments; it will not generally accelerate an arbitrary substring search. That distinction is worth observing rather than hiding behind a fast-looking interface.

Try this in `psql` after the import:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT title_id, primary_title
FROM imdb_course.titles
WHERE start_year = 1994
ORDER BY primary_title;
```

Check the plan and actual rows returned. A small or unselective sample may lead to a sequential scan. `EXPLAIN ANALYZE` runs the query, so use it here with the supplied read-only statement. (5)

Connect a Small Streamlit Dashboard

```
streamlit run app.py --server.address 0.0.0.0
```

Open the forwarded web port, normally 8501, from the Codespace. Forward the Streamlit port only. PostgreSQL stays on loopback.

Search has By Name, By Year and By Rating tabs. These list individual movies, with links to their notes. Reports holds the grouped charts. Change the minimum vote count and the search controls, then inspect the returned movies. An empty result means no qualifying movie was found in this sample; it is not necessarily a connection problem.

The data layer opens a connection for each query and closes it afterward. Streamlit caches returned data for 60 seconds. It does not cache one shared database connection. The refresh button clears cached results. This keeps resource ownership simple while showing that cached displays can lag behind database changes. Streamlit distinguishes cached data from cached resources. (6)

Psycopg passes query values separately from the SQL text:

```
connection.execute(
    "SELECT primary_title FROM imdb_course.titles WHERE start_year = %s",
    (1994,),
)
```

The `%s` is a driver parameter marker. Do not interpolate the value into the SQL string yourself. This is the same habit as parameterized SQLite queries, with a different driver interface. (7)

Add Grouping and Aggregation

The **By year** tab groups joined title/rating rows:

```
SELECT t.start_year, count(*) AS movies,
       round(avg(r.average_rating), 2) AS mean_rating
FROM imdb_course.titles t
JOIN imdb_course.ratings r USING (title_id)
WHERE t.start_year IS NOT NULL AND r.num_votes >= 1000
GROUP BY t.start_year
ORDER BY t.start_year;
```

`GROUP BY` reduces many movie rows to one row per year. `count(*)` counts movies that survived the join and vote threshold. It does not count user votes. The average gives each title's published rating equal weight; it is not an average calculated from individual users' votes.

The dashboard displays the result as a table and a bar chart. Lower the vote threshold and observe which years gain titles. Change a SQL query in `database.py`, restart the app or clear its cache, and inspect the resulting display.

Add Window Functions

A window function calculates across related rows while retaining an output row for each input row. `PARTITION BY` identifies the group used for that calculation, and the `ORDER BY` inside `OVER` supplies its ordering. Window functions are useful for rankings and comparisons without first collapsing everything into one summary row. (8)(9)

The `ranked_movies` query uses this expression:

```
dense_rank() OVER (
    PARTITION BY t.start_year
    ORDER BY r.average_rating DESC
) AS year_rank
```

The rank starts again for each year. Equal ratings share a rank. `dense_rank` leaves no gaps between successive rank values. The query wraps this calculation in a common table expression named `ranked`, then selects `year_rank <= 3` in the outer query. We need that second query level because a window result is not available to the same level's `WHERE` clause. (8)

This selects the top three distinct rating levels, which may include more than three movies. The dashboard caps the displayed result at 300 rows. An ordering by vote count and title makes the displayed ties easier to inspect without changing the rating-based rank.

The Reports page first groups movies by year, then applies windows to those yearly totals:

```
lag(movies) OVER (ORDER BY start_year)
```

`lag` reads the preceding row's value. Subtracting it from the current count gives the change between adjacent years present in the query result. If a year is missing from the sample, the previous row is not necessarily the previous calendar year. The tab includes `previous_year` so that gap is visible.

The running total uses an explicit frame:

```
sum(movies) OVER (
    ORDER BY start_year
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

This adds the counts from the first yearly row through the current row. On the first row, the change is `NULL` because there is no preceding count, but the running total is already defined. (8)

Accounts, Permissions, and Recovery

The three dashboard lectures leave us with a useful application. This short follow-up adds the server responsibilities that SQLite did not require us to manage separately. It can use the same imported data; there is no second dataset to load.

The setup account and the application account

The Codespace's Linux account owns the server process and its files. PostgreSQL roles live inside the database server. They are separate accounts, even when their names happen to match. A role with `LOGIN` can connect; roles can also own objects and receive privileges. (10)

Our startup script uses `initdb -U student`. That creates the initial PostgreSQL superuser, not an ordinary student application account. It owns the exercise database and imported schema. Keep it for setup and recovery. Give the analytics page a read-only login. The transactional page gets a separate login with write access limited to movie notes.

Account	Purpose	Powers in this exercise
Codespace Linux user	Start the process and manage its files	Operating-system file permissions
`student` PostgreSQL role	Import data, create indexes, manage roles	Initial database superuser
`imdb_reader` PostgreSQL role	Run dashboard queries	Connect, use the course schema, select its tables
`imdb_writer` PostgreSQL role	Manage movie notes	Read the sample; insert, update and delete notes only

From the `code` directory, activate the existing virtual environment. Keep the administrative connection while creating the application role:

```
export DATABASE_URL=postgresql://student@127.0.0.1:5433/imdb
python setup_roles.py
python check_permissions.py
```

The setup script creates `imdb_reader` with login permission and no administrative attributes. It grants database `CONNECT`, schema `USAGE`, and table `SELECT`. It also supplies default table privileges for future tables created by the same owner. Default privileges are specific to the creating role; they do not automatically cover tables created by every administrator. (10)

The checker connects as `imdb_reader`. Reading titles must work. An update must be denied, even though its `WHERE false` condition would change no rows. Creating another database role must also be denied. Each attempted operation is rolled back, including an unexpectedly successful operation.

Now run the actual dashboard using the application login:

```
export DATABASE_URL=postgresql://imdb_reader@127.0.0.1:5433/imdb
export TRANSACTION_DATABASE_URL=postgresql://imdb_writer@127.0.0.1:5433/imdb
streamlit run app.py
```

Searches, aggregates, and window queries should all work. Importing, replacing tables, and creating indexes remain administrative operations. Before running those scripts, switch the connection back to `student`. Replacing the imported schema removes its grants, so run `setup_roles.py` again afterward.

Three pages, two application accounts

Use the sidebar's `Page` control to choose `Search`, `Notes` or `Reports`. Search lists movies by name, year or rating range. Reports shows ratings and counts as charts. Its queries use `DATABASE_URL`, configured above as `imdb_reader`.

Notes uses `TRANSACTION_DATABASE_URL`, configured as `imdb_writer`. The setup script creates a `movie_notes` table with at most one note per imported title. Its title identifier is a foreign key, and a primary key prevents duplicate notes for the same movie. Notes must contain 1 to 2000 nonblank characters. The writer can read IMDb tables but can change only the notes table. Table privileges, rather than the visibility of a button, enforce that restriction. (10)

Page	Connection	Allowed work
Search	`DATABASE_URL`, reader	Read movie data and calculate summaries
Notes	`TRANSACTION_DATABASE_URL`, writer	Create, read, update and delete movie notes

Open the Movie notes link on a search result. The notes page carries the title and identifier, so no copying is needed. Create a note, edit its text, then select the deletion confirmation and delete it. Each write uses its own database transaction and commits only on success; an error rolls it back. These are database accounts selected by the application page, not separate human logins.

`notes_page.py` contains the controls. `notes_store.py` contains parameterized SQL and owns the writer connection. The analytics layer keeps its own read-only connection. Imported IMDb records remain unchanged throughout the exercise.

Run `python check_transaction_page.py` with the reader and writer environment settings to repeat a live check of the page. It exercises create, update and delete, rejects a duplicate note, and verifies permission boundaries. It needs the first imported title to have no existing note and removes the test note afterward. Replacing the imported schema with `import_imdb.py --replace` also removes movie notes; back up first, then rerun account setup after import.

Listening, authentication, and permissions

These are three different checks. `listen_addresses` determines which network interfaces accept connections. `pg_hba.conf` chooses a client authentication rule. SQL grants determine what the connected role may do. PostgreSQL uses the first matching authentication record. (11)

```
SHOW listen_addresses;
SHOW port;
SHOW config_file;
SHOW hba_file;
SHOW data_directory;
```

Run these as the administrative role to locate the actual configuration. The exercise should listen on `127.0.0.1` and port 5433. Its log is `.private-postgres/server.log`.

This private cluster uses `trust` authentication. A local client can claim an existing role without a password, including `student`. The permission experiment demonstrates SQL authorization; it does not protect against another program running in the same Codespace choosing the administrator's name. This is why the database listener stays local and its port stays unforwarded. Loopback binding is not a separate firewall rule.

A shared server needs password or another suitable authentication method, carefully scoped client rules, and a network boundary. Remote connections also need transport encryption, commonly Transport Layer Security (TLS). Setting a role password by itself does not replace a matching `trust` rule. Keep credentials out of source files and repositories. (11)

Process management depends on the environment

Environment	What manages the server
This Codespace	Supplied script starts a private cluster with `pg_ctl`
A normal Linux host	Often a package-managed service
Windows	Often a Windows service
macOS	Depends on the installation; a service manager or `pg_ctl`

The SQL role and privilege ideas carry across these systems. Installation paths and service commands do not. This exercise supplies the Ubuntu/Codespace procedure rather than claiming to include complete Windows and macOS installation walkthroughs.

To stop only this exercise cluster from its `code` directory:

```
"$(pg_config --bindir)/pg_ctl" -D "$PWD/.private-postgres" -m fast -w stop
```

Run `bash start-postgres.sh` to start it again. The stored data and database roles remain in the private cluster directory.

A backup is useful when a restore works

Switch back to the administrative connection and run:

```
export DATABASE_URL=postgresql://student@127.0.0.1:5433/imdb
python backup_restore.py
```

The script uses `pg_dump` to create a custom-format archive of the `imdb_course` schema. It creates a new database with a unique `imdb_restore_` name, then restores the archive there with `pg_restore`. It never restores over the working `imdb` database. Both client tools should match the installed server's major version. (12)(13)

It checks row counts and content digests for the imported tables and for movie notes when that table exists, then compares index definitions. Expect a passing check for each table and for the indexes and the names of the archive and restored database. The database is left available for inspection; each run creates another separate copy. Backups are stored under the ignored `.private-backups` directory.

`pg_dump` backs up one database, not the server's login roles. This exercise restores with ownership and access grants omitted, so the administrative restoring role owns the objects. Reapplying application permissions is a separate step. A full recovery plan must account for roles and configuration as well as table contents. A backup in the same Codespace also disappears when that Codespace is deleted. (12)

Use the printed restore database name in `psql`, then inspect its tables. When finished, remove that particular test database with `dropdb`; check its name carefully. Keep the original `imdb` database.

Worked example: minimum movie length

Compare yearly movie counts and average ratings after leaving out movies below a selected length. The control changes which movies contribute to each year's report. It does not change the stored data.

Step 1. In `database.py`, replace `yearly_summary` with:

```
def yearly_summary(minimum_votes=1000, minimum_movie_length=0):
    return query("""
        SELECT t.start_year, count(*) AS movies,
               round(avg(r.average_rating), 2) AS mean_rating
        FROM imdb_course.titles t
        JOIN imdb_course.ratings r USING (title_id)
        WHERE t.start_year IS NOT NULL
          AND r.num_votes >= %s
          AND t.runtime_minutes >= %s
        GROUP BY t.start_year ORDER BY t.start_year
    """, (minimum_votes, minimum_movie_length))
```

The source column is named `runtime_minutes`; it holds movie length in minutes. The two `%s` placeholders match the two values in the parameter tuple, in order. `WHERE` filters individual movies before `GROUP BY` calculates each year's count and average. Movies with an unknown length (`NULL`) fail the comparison, even when the minimum is zero. The default argument keeps existing one-argument calls valid.

Step 2. In `app_basics.py`, add this control immediately after the minimum-votes input:

```
minimum_movie_length = st.sidebar.number_input(
    'Minimum movie length (minutes)',
    min_value=0, value=0, step=10,
)
```

Step 3. Replace the cached `summary` function in `app_basics.py` with:

```
@st.cache_data(ttl=60)
def summary(votes, minimum_movie_length):
    return database.yearly_summary(votes, minimum_movie_length)
```

Streamlit uses function arguments in its cache key. A result for 90 minutes must not be reused for 120 minutes.

Step 4. Inside the `with second:` block, replace the call with:

```
rows = summary(votes, minimum_movie_length)
```

Keep the existing table and bar chart below it. Change their caption to `Counts reflect this sample, vote threshold, and minimum movie length; ratings give each title equal weight.`

Step 5. Save both files and run `streamlit run app_basics.py --server.address 0.0.0.0` from the code directory, or let the running app reload. Open **By year**. Hold minimum votes fixed and compare 0, 90, and 120 minutes. Raising the minimum cannot increase a year's count. A year can disappear when no movies qualify. Its average rating can rise or fall because different movies contribute to it. Returning to an earlier cutoff should reproduce its results while the data stays unchanged.

Your turn: maximum movie length

Add a **Maximum movie length (minutes)** control to `app_basics.py` and a `maximum_movie_length` parameter to `database.yearly_summary`. Pass the value through the cached `summary` function and its call. Apply an inclusive upper bound to the movie-length column before grouping. Keep the existing minimum and vote filters.

Use 300 minutes as the control's initial maximum. Keep existing calls valid by making the data-layer maximum optional, with no upper bound when omitted. When a user chooses a maximum below the minimum, display a clear message and skip the summary query.

Check 90-120 minutes and then 90-180 minutes with votes held fixed. Widening the interval cannot reduce a year's count. Test equal limits too: 120-120 should include only movies exactly 120 minutes long. Confirm that changing either limit updates the table and chart, and that an empty result is handled without an error.

Practice

1. Run the downloader or reuse your existing archives. Prepare the 1975-1999 movie selection and record the two row counts printed by the importer. Check the counts independently with `psql`.
2. In `database.py`, change the search ordering from most votes to highest rating. Keep a deterministic final ordering by identifier. Confirm the Search tab changes accordingly.
3. Complete the maximum movie length extension in the worked example. Explain why both bounds belong in `WHERE` and why both values must reach the cached function.
4. Change `dense_rank` to `row_number` and select the first three rows per year. Include vote count and identifier in the window ordering. Compare the handling of ties.
5. Add a windowed average rating for each year beside each movie's own rating. Inspect the difference between the two values in the query results.
6. Find a missing year in a small sample, or construct a small temporary example. Explain why `lag` can compare nonconsecutive calendar years.
7. Run the year-filter plan before and after its secondary index exists. Record the actual plan rather than assuming the index must be selected. Re-run `indexes.sql` to restore the exercise index afterward.
8. Run the application permission checks. Explain why a denied update and password-free local authentication can both be true.
9. Restore a backup into a separate database. Verify the data and indexes; identify which account information the archive does not contain.

Considerations

- Which parts of the application changed when storage moved from SQLite to PostgreSQL? Which parts of the dashboard could stay the same?
- What conclusions about film history would be unreliable when using movies from 1975-1999 rather than a random sample or the full dataset?

- Should a ranking threshold use the number of votes, the average rating, or both? What kinds of films disappear under each choice?
- When is a one-minute result cache acceptable for a dashboard? What would change for a screen showing rapidly changing transactions?
- Why might the average of title ratings answer a different question from an average across all individual user ratings?

References

1. Wikipedia contributors. “PostgreSQL.” https://en.wikipedia.org/wiki/PostgreSQL
2. IMDb. “IMDb Non-Commercial Datasets.” Dataset fields, missing values, updates, and terms. https://data.imdb.com/non-commercial-datasets/
3. Ubuntu. “Install and configure PostgreSQL.” Ubuntu Server documentation. https://documentation.ubuntu.com/server/how-to/databases/install-postgresql/
4. PostgreSQL Global Development Group. “COPY.” PostgreSQL documentation. https://www.postgresql.org/docs/16/sql-copy.html
5. PostgreSQL Global Development Group. “Using EXPLAIN.” PostgreSQL documentation. https://www.postgresql.org/docs/16/using-explain.html
6. Streamlit. “Caching overview.” https://docs.streamlit.io/develop/concepts/architecture/caching
7. Psycopg contributors. “Passing parameters to SQL queries.” Psycopg 3 documentation. https://www.psycopg.org/psycopg3/docs/basic/params.html
8. PostgreSQL Global Development Group. “Window Functions.” PostgreSQL tutorial. https://www.postgresql.org/docs/16/tutorial-window.html
9. Wikipedia contributors. “Window function (SQL).” https://en.wikipedia.org/wiki/Window_function_(SQL)
10. PostgreSQL Global Development Group. “Database Roles” and “Privileges.” PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/user-manag.html and https://www.postgresql.org/docs/16/ddl-priv.html
11. PostgreSQL Global Development Group. “The pg_hba.conf File.” PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/auth-pg-hba-conf.html
12. PostgreSQL Global Development Group. “pg_dump.” PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/app-pgdump.html
13. PostgreSQL Global Development Group. “pg_restore.” PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/app-pgrestore.html